

Pętle i tablice w Java

Zbigniew Marszalek

Instrukcja while

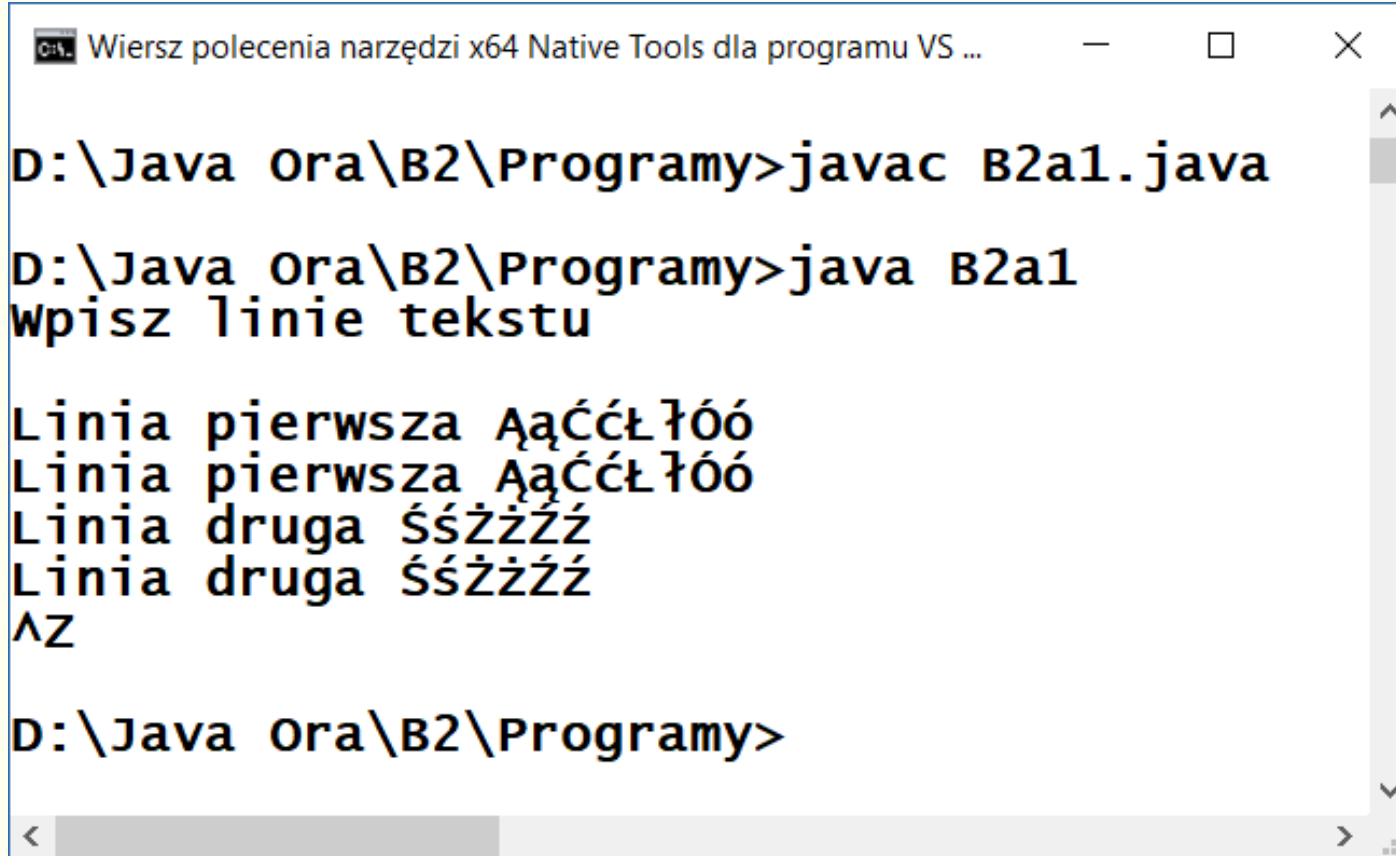
Na początku przedstawmy instrukcję *while()*, która jest w pewien sposób iteracyjnym wykonywaniem zadanej operacji, aż do momentu spełnienia określonego kryterium. Postać instrukcji *while()* jest następująca

```
while(wyrażenie logiczne)
{ ciało pętli }
```

Zadanie. Należy napisać program powtarzający wprowadzane linie tekstu z konsoli do momentu podania przez użytkownika końca zbioru CTR+Z (widoczne jako ^Z) – dla systemu Windows)

```
1 import java.io.*;
2 import java.util.*;
3 public class B2a1 {
4     public static void main(String[] args) throws IOException {
5         System.setProperty("file.encoding", "Cp852");
6         // Ustawienie kodowania programu.
7         BufferedReader we = new
8             BufferedReader(new InputStreamReader(System.in, "Cp852")) ;
9         // Utworzenie buforowanego strumienia wejsciowego (strona kodowa Cp852)
10        PrintWriter wy = new
11            PrintWriter(new OutputStreamWriter(System.out, "Cp852"), true) ;
12        // Utworzenie buforowanego strumienia wyjsciowego (strona kodowa Cp852)
13        String s;
14        // Deklaracja łańcucha znaków
15        wy.println("Wpisz linie tekstu\n");
16        // Wypisanie tekstu.
17        while ((s = we.readLine()) != null)
18            //Wprowadzenie nowej linii i sprawdzenie, czy jest to koniec zbioru.
19            wy.println(s);
20        //Wypisanie linii tekstu.
21    }
22 }
```

Wykonanie programu w konsoli



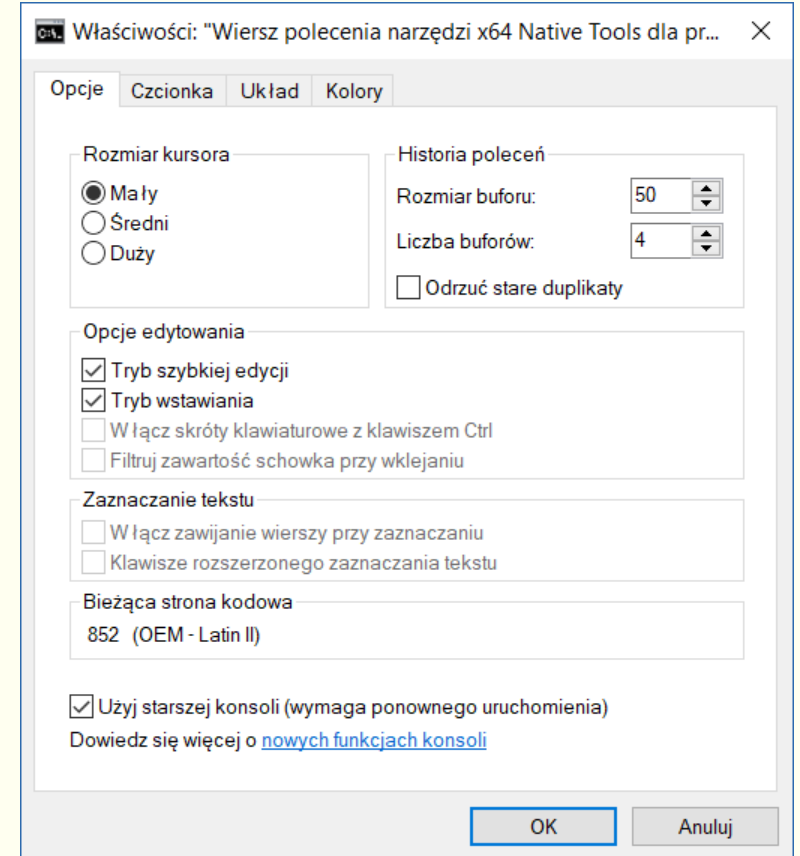
```
Wiersz polecenia narzędzi x64 Native Tools dla programu VS ...

D:\Java Ora\B2\Programy>javac B2a1.java

D:\Java Ora\B2\Programy>java B2a1
wpisz linie tekstu

Linia pierwsza AąĆćŁóó
Linia pierwsza AąĆćŁóó
Linia druga ŚśŻżŻż
Linia druga ŚśŻżŻż
^Z

D:\Java Ora\B2\Programy>
```



Właściwości: "Wiersz polecenia narzędzi x64 Native Tools dla pr..."

Opcje Czcionka Układ Kolory

Rozmiar kursora

- ☒ Mały
- ☐ Średni
- ☐ Duży

Historia poleceń

Rozmiar buforu: 50

Liczba buforów: 4

☐ Odrzuć stare duplikaty

Opcje edytowania

- ☒ Tryb szybkiej edycji
- ☒ Tryb wstawiania
- ☐ Włącz skróty klawiaturowe z klawiszem Ctrl
- ☐ Filtruj zawartość schowka przy wklejaniu

Zaznaczanie tekstu

- ☐ Włącz zawijanie wierszy przy zaznaczaniu
- ☐ Klawisze rozszerzonego zaznaczania tekstu

Bieżąca strona kodowa

852 (OEM - Latin II)

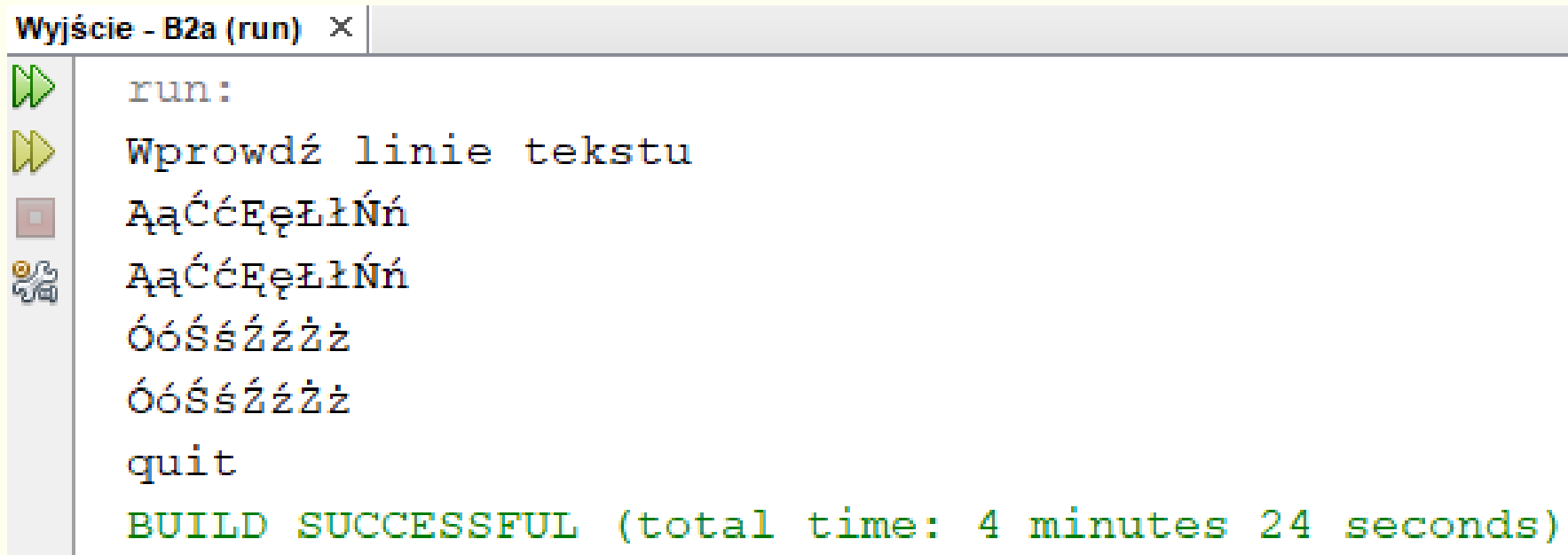
☒ Użyj starszej konsoli (wymaga ponownego uruchomienia)

Dowiedz się więcej o [nowych funkcjach konsoli](#)

OK Anuluj

```
package b2a;
import java.util.*;
public class B2a {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in, "Cp1250");
// Deklaracja skanera tekstu.
        System.out.println("Wprowadź linie tekstu");
// Wypisanie komunikatu.
        String s;
// Deklaracja łańcucha znaków.
        while (!(s = in.nextLine()).equals("quit"))
//Wprowadzenie nowej linii i sprawdzenie, czy
//jest to koniec zbioru.
            System.out.println(s);
//Wypisanie linii tekstu.
    }
}
```

Otrzymywane wyniki

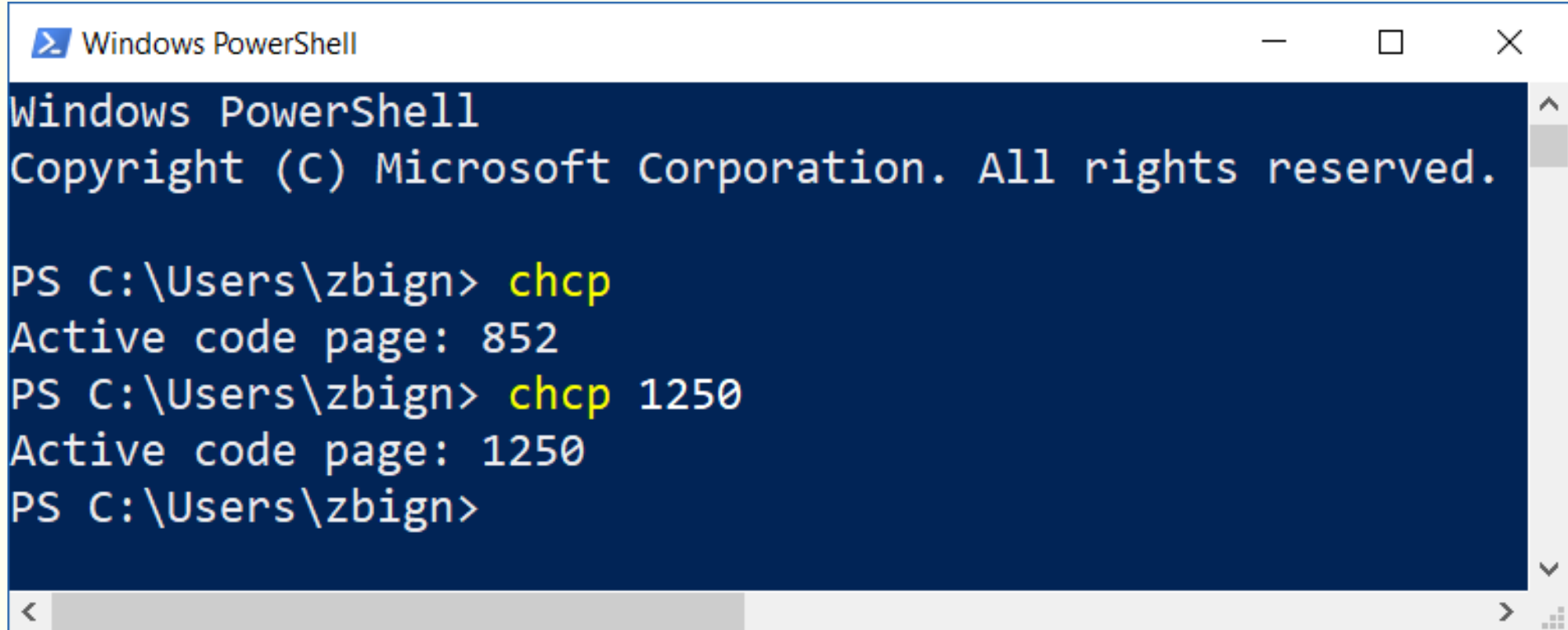


```
Wyjście - B2a (run) x
run:
Wprowadź linie tekstu
AąĆćĘęŁłŃń
AąĆćĘęŁłŃń
ÓóŚśŻżŻż
ÓóŚśŻżŻż
quit
BUILD SUCCESSFUL (total time: 4 minutes 24 seconds)
```

Uwaga. Zawsze należy rozpoznać systemy kodowania znaków strumienia wejściowego i wyjściowego programu w celu dostosowania metod do porwanego wyświetlania znaków.

Zmiana kodowania konsoli

W celu zmiany kodowania na stronę polską należy wydać polecenie `chcp` podając numer strony kodowej : `chcp 1205` .



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\zbign> chcp
Active code page: 852
PS C:\Users\zbign> chcp 1250
Active code page: 1250
PS C:\Users\zbign>
```

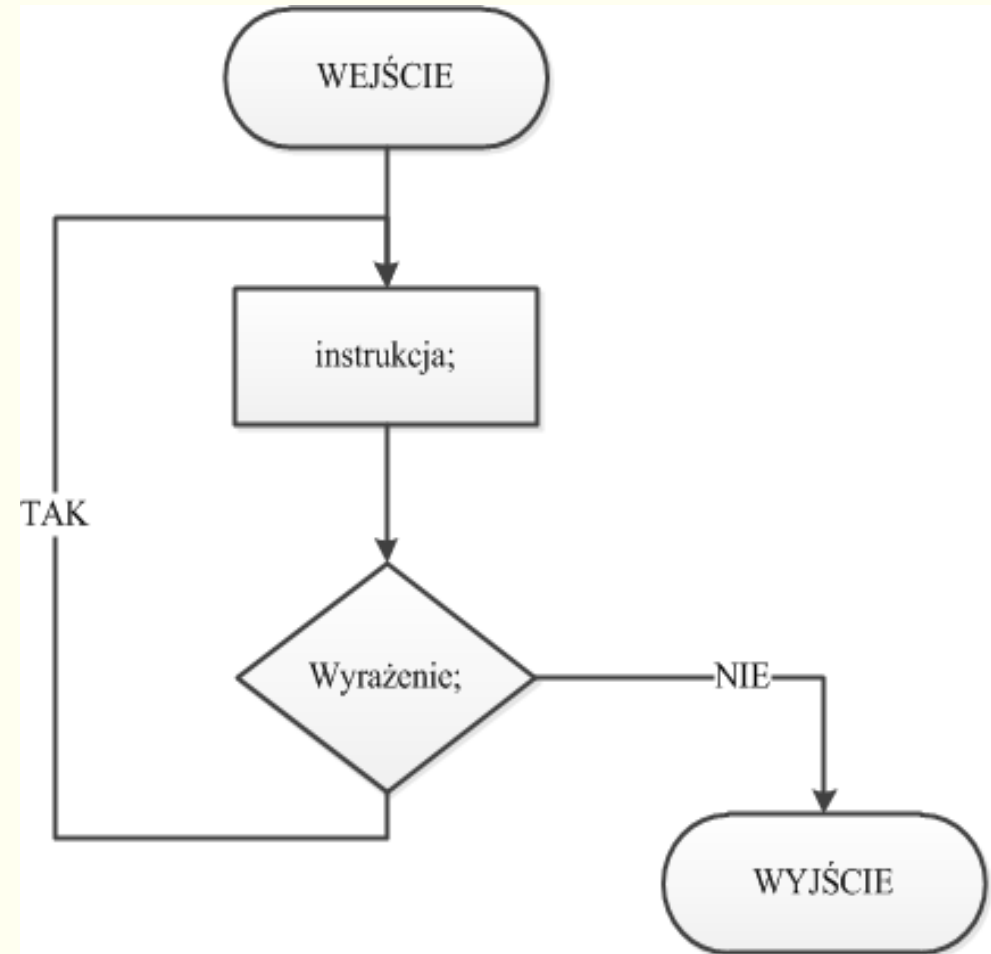
Pętla do...while

Nie jest to jednak instrukcja stworzona bezpośrednio na potrzeby programistów korzystających z języka C#. Składnia tej instrukcji została zaczerpnięta z języka Fortran, w którym warunek zatrzymania pętli był sprawdzany dopiero na końcu iteracji. Formalnie instrukcja ma następującą składnię:

do

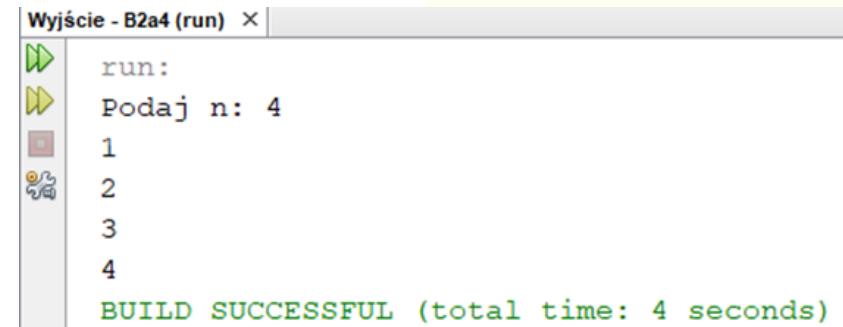
{ ciało pętli }

while(wyrażenie logiczne);



Zadanie. Napisać program wypisujący kolejne liczby naturalne z wykorzystaniem pętli do...while.

```
Scanner in = new Scanner(System.in) ;
int j = 0;
//Inicjowanie zmiennej wartością zero.
System.out.print("Podaj n: ");
int n = in.nextInt();
    if (n > 0)
//Sprawdzenie poprawności wprowadzonej liczby
{
    do
        System.out.println(++j);
/* Przyrostkowa postać inkrementacji powoduje
zmniejszenie albo zwiększenie wartości zmiennej
przed wzięciem jej do obliczeń */
    while (--n > 0);
```



```
Wyjście - B2a4 (run) x
run:
Podaj n: 4
1
2
3
4
BUILD SUCCESSFUL (total time: 4 seconds)
```

Operatory przypisania

W języku C# jest możliwe przypisywanie wartości poprzez operatory jedno i wieloargumentowe. Dla arytmetycznych operatorów dwuargumentowych omówionych wcześniej +, -, *, /, % możemy stosować operator przypisania w następujący sposób:

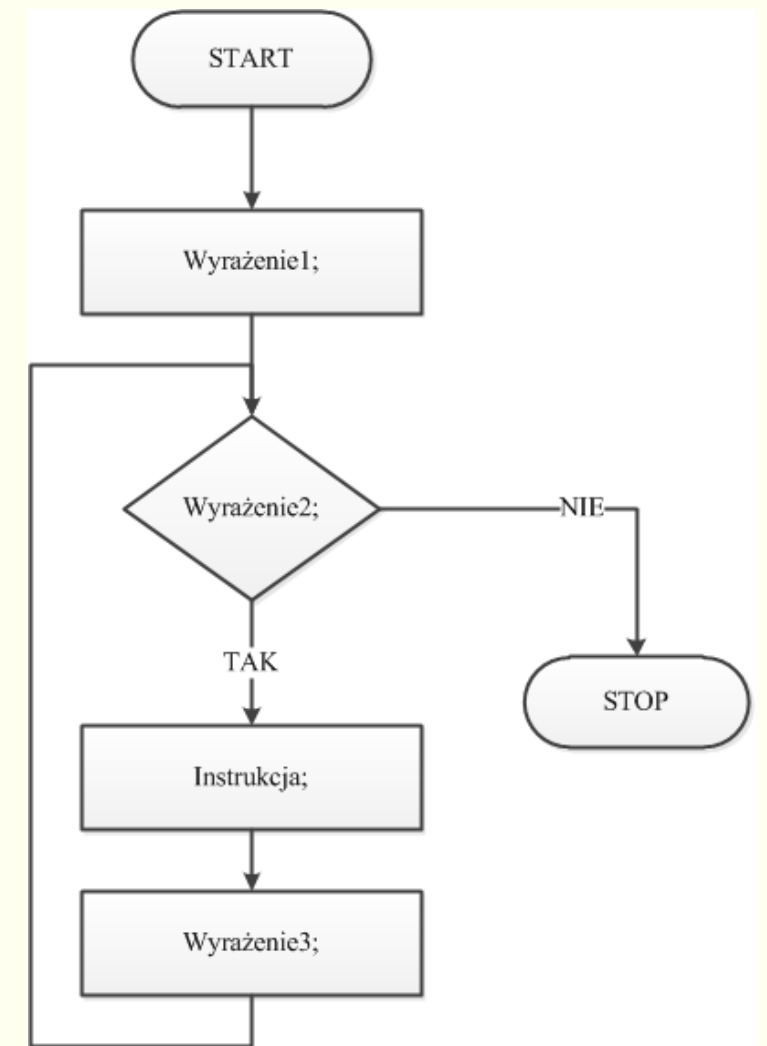
`zmienna (operator) = wyrażenie;`
jest równoważne
`zmienna = zmienna (operator) wyrażenie;`

np.: `i=i+3`; możemy zapisać `i+=3`; jednocześnie instrukcja `x*= a-3`; odpowiada instrukcji `x=x*(a-3)`;

Instrukcje zwiększania i zmniejszania wartości zmiennych oznaczamy ++ lub --. Operator zwiększania ++ dodaje jeden, natomiast operator zmniejszania -- odejmuje 1. Mogą one mieć postać przedrostkową (przed zmienną: ++*i*) oraz przyrostkową (po zmiennej: *i*++). W przypadku inkrementacji albo dekrementacji przedrostkowej zwiększana albo zmniejszana jest wartość zmiennej przed użyciem jej w wyrażeniu. Natomiast inkrementacja przyrostkowa powoduje zmianę wartości zmiennej dopiero po wykonaniu zadanych operacji.

Pętla for

Najpopularniejszą deklaracją powtarzania operacji jest pętla *for()* mająca następujący schemat budowy.



```
for( Wyrażenie1; Wyrażenie 2; Wyrażenie3 )  
{ ciało pętli }
```

Zadanie. Napisać program wypisujący drukujący tabliczkę mnożenia.

```
    for (int i = 1; i <= 10; i++)  
//Pętla iteracyjna numerująca wiersze  
    {  
        for (int j = 1; j <= 10; j++)  
//Pętla iteracyjna numerująca kolumny  
            System.out.printf("%4d", i * j);  
//Wypisanie wyniku mnożenia  
            System.out.println();  
//Oddzielenie wierszy  
    }
```

run:

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

Pętla for-each

Składnia:

```
for( typ identyfikator : kolekcja )  
{ ciało pętli }
```

Wyrażenie *typ identyfikator* deklaruje zmienną iteracyjną (ang. iteration variable). Taka konstrukcja jest tworzona tylko raz dla każdego elementu w danej kolekcji i reprezentuje bieżącą wartość elementu kolekcji. Zmienna iteracyjna dostępna jest tylko do odczytu i próba nadanie jej wartości zakończy się błędem kompilatora. Postępując w myśl tej zasady postaramy się stopniowo przedstawiać możliwości programowania w języku Java. Na początku wykorzystamy program ze wstępu książki i uzupełnimy go w celu wykorzystania pętli for-each do wyświetlenia argumentów przekazanych do funkcji main().

```
package b2a6;

public class B2a6 {
    public static void main(String[] args) {
        for(String s: args)
            System.out.println(s);
        //Pętla wyświetla argumenty metody main()
    }
}
```

Wyjście - B2a6 (run) ×



run:



Arg1



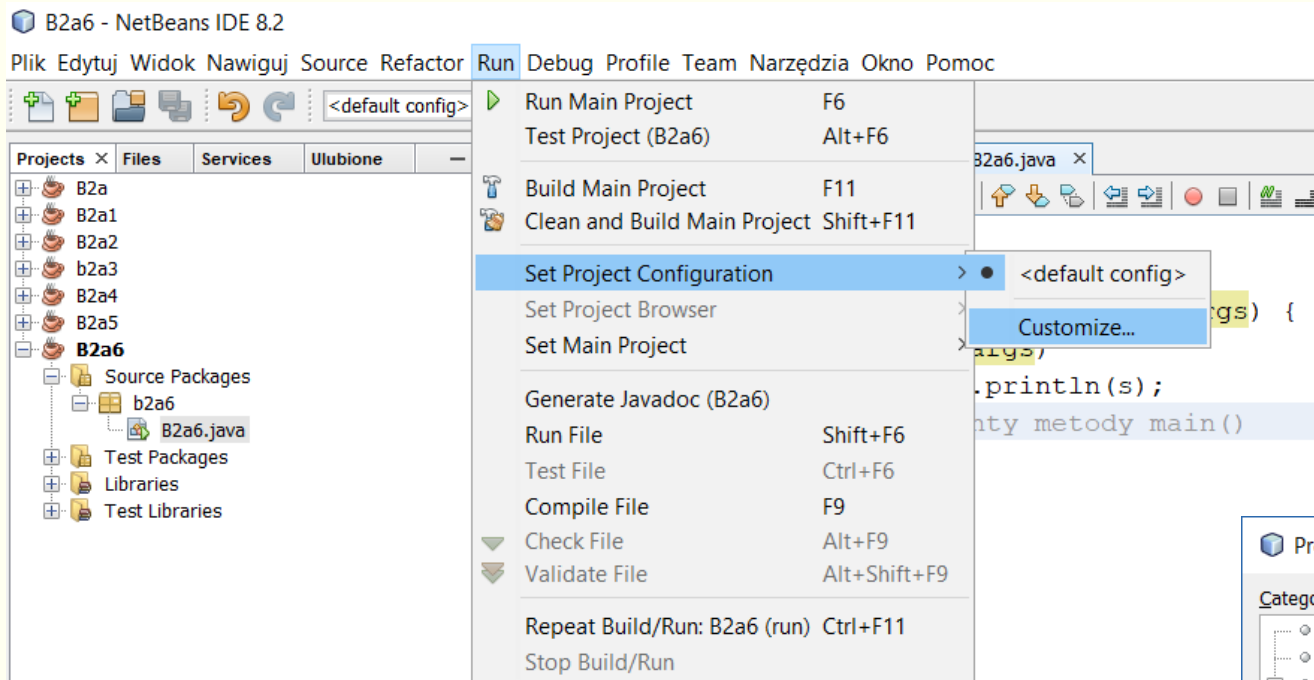
Arg2



BUILD SUCCESSFUL (total time: 0 seconds)

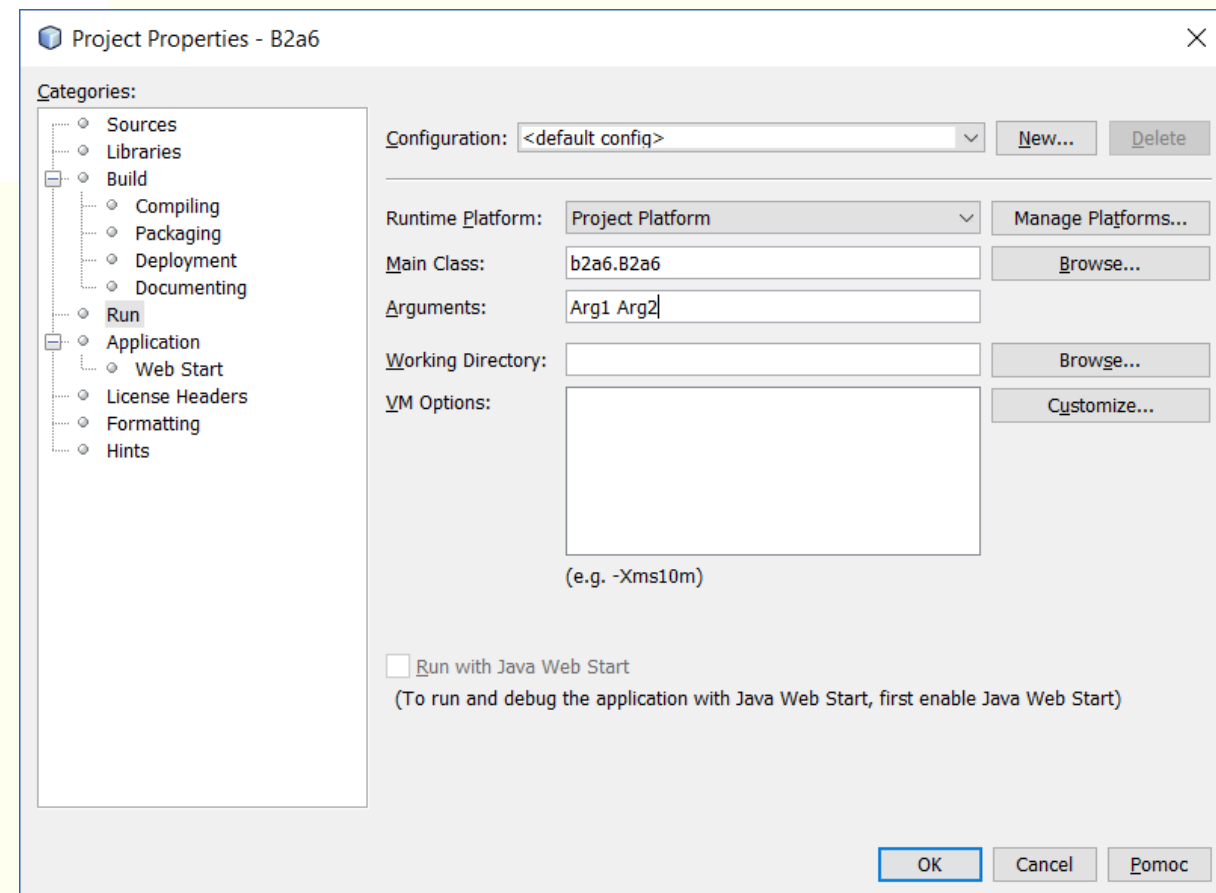
Program ten demonstruje pobranie przez metodę `main()` listy argumentów i zwrócenie liczby argumentów. Pętla `for-each` pobiera z tablicy `args` kolejne argumenty i wyświetla je na konsoli korzystając z metody `println()` pochodzącej z klasy `System.out`.

W celu wprowadzenia argumentów do pakietu NetBeans IDE wybieramy z menu pozycję `Run/Set Main Project`, co pokazuje Rysunek. Następnie klikamy na `Customize` i w oknie `Arguments` wpisujemy argumenty uruchomienia programu. W naszym programie wpisaliśmy: `Arg1 Arg2`.



Rysunek. Zakładka
właściwości projektu w
NetBeans IDE

Okno ustawień opcji
debugowania w NetBeans IDE



Słowo kluczowe break

Słowo kluczowe break pozwala na wcześniejsze przerwanie pętli.

```
1 package b2a7;  
2 public class B2a7 {  
3     public static void main(String[] args) {  
4         int i = 1;  
5         do  
6         {  
7             System.out.println(i++);  
8             if (i > 4) break;  
9             //Wyjście z pętli, gdy zmienna i przekroczy wartość 4  
10        }  
11        while (true);  
12    }  
13 }  
14 }
```

Wyjście - B2a7 (run) ×

run:
1
2
3
4
BUILD SUCCESSFUL (total time: 0 seconds)

Instrukcja break zamiast goto

Język Java nie posiada tradycyjnej wersji instrukcji goto z języka C, ze względu na obiektową składnię języka. Uniemożliwia to tworzenie programów o niebezpiecznym kodzie przekazującym sterowanie w dowolne miejsce. Istnieją jednak sytuacje, w których wskazane jest opuszczenie głęboko zagnieżdżonych pętli. Dlatego Java definiuje rozszerzoną wersję instrukcji break umożliwiającą pominąć kilka bloków kodu. Instrukcja dodatkowo określa miejsce, do którego ma zostać przekazane sterowanie programu za pomocą etykiety. Ogólna postać instrukcji break z etykietą jest następująca:

break etykieta;

Poniższy przykład zawiera trzy zagnieżdżone bloki z przypisanymi nazwami. Instrukcja break powoduje przejście do następnej instrukcji programu po końcu bloku drugi z pominięciem instrukcji *println()*.

```
boolean t = true;
    pierwszy:{
        drugi:{
            trzeci:{
                System.out.println("Przed break");
                if(t) break drugi;
                // Wyjście poza drugi blok.
                System.out.println("To się nie wykona");
            }
            System.out.println("To się wykona");
        }
        System.out.println("Po drugim bloku");
    }
```

Wyjście - B2a8 (run) ×



run:



Przed break



Po drugim bloku



BUILD SUCCESSFUL (total time: 0 seconds)

Instrukcja continue

Jeżeli zachodzi konieczność pominięcia kodu programu w instrukcji pętli, ale dalsze wykonywanie iteracji, to możemy użyć instrukcji continue. Taka operacja jest równoważna przeniesieniu sterowania tuż przed koniec pętli. Poniżej fragment programu drukujący tabliczkę mnożenia ilustruje wykorzystanie polecenia continue.

```
        for (int i = 1; i <= 10; i++)  
//Pętla iteracyjna numerująca wiersze  
        for (int j = 1; j <= 10; j++) {  
//Pętla iteracyjna numerująca kolumny  
            System.out.printf("%4d", i * j);  
//Wypisanie wyniku mnożenia  
            if(j % 10 != 0) continue;  
            System.out.println();  
        }
```

Instrukcja Switch

Składnia instrukcji switch jest następująca:

```
switch( wyrażenie){  
    case wyrażenie :  
        instrukcje  
        break;  
  
    ...  
  
    default :  
        instrukcje  
        break;  
  
}
```

Zadanie. Należy napisać program zamieniający dzień tygodnia podany za pomocą liczby na nazwę słowną.

```
Scanner in = new Scanner(System.in);
System.out.print("Podaj dzień tygodnia :");
    int t = in.nextInt();
    String dzien;
    switch (t)
    {
        case 1:
            dzien = "poniedziałek";
            break;
        case 2:
            dzien = "wtorek";
            break;
        case 3:
            dzien = "środa";
            break;
        case 4:
            dzien = "czwartek";
            break;
```

```
case 5:
    dzien = "piątek";
    break;
case 6:
    dzien = "sobota";
    break;
case 7:
    dzien = "niedziela";
    break;
default :
    dzien = "nieznany dzień tygodnia";
    break;
}
System.out.println("Dniem tygodnia jest "+dzien);
```

Wyjście - B3 (run) ×

```
run:
Podaj dzień tygodnia :1
Dniem tygodnia jest poniedziałek
BUILD SUCCESSFUL (total time: 4 seconds)
```

Wyjście - B3 (run) ×

```
run:
Podaj dzień tygodnia :0
Dniem tygodnia jest nieznany dzień tygodnia
BUILD SUCCESSFUL (total time: 4 seconds)
```

Tablice

Tablice są indeksowanym ciągiem elementów tego samego typu. Tablice jednowymiarowe są deklarowane za pomocą składni:

```
Type [ ] nazwa_tablicy
```

Poniższy program ilustruje sposób deklaracji tablicy liczb zmiennoprzecinkowych, łańcuchów znakowych i inicjalizacji tablicy liczb całkowitych.


```
    double [] a;  
// Deklaracja uchwytu do tablicy liczb zmiennoprzecinkowych.  
    String [] b;  
// Deklaracja uchwytu do tablicy łańcuchów znakowych.  
    int [] c = { -1, 0, 4};  
// Deklaracja tablicy liczb całkowitych z inicjalizacją tablicy.  
    for (int i = 0; i < c.length; i++)  
    {  
        System.out.printf("%d ", c[i]);  
// Wydruk liczb zapisanych w tablicy za pomocą pętli for.  
        System.out.println();  
// Przejście do nowej linii.  
        for(int x: c)  
        {  
            System.out.printf("%d ", x);  
// Wydruk liczb zapisanych w tablicy za pomocą instrukcji  
// for-each.  
            System.out.println();  
        }  
    }  
}
```

```
Wyjście - B4 (run) x
run:
-1 0 4
-1 0 4
BUILD SUCCESSFUL (total time: 0 seconds)
```

Wydruk liczb zapisanych w tablicy został wykonany za pomocą pętli `for` i instrukcji `foreach`. Warto podkreślić, że obiekt `tablica` przechowuje ilość zapisanych w nim elementów. Użytkownik może tylko odczytać ilość przechowywanych elementów w tablicy za pomocą odpowiedniej właściwości, ale nie może dokonywać zmian w wielkości przechowywanych elementów przez obiekt. Może natomiast zmieniać zapisane wartości w tablicach zgodnie z zaprojektowanymi algorytmami.

Do konstrukcji tablicy używamy słowa kluczowego `new`. Poniżej podajemy przykłady konstrukcji tablic:

```
        Locale pl = new Locale("pl","PL");
        int[] a = new int []{ -5, 0, 10, 6};
// Konstrukcja tablicy liczb całkowitych za pomocą new.
    for (int i = 0; i < a.length; i++)
        System.out.println(a[i]);
// Wydruk zawartości tablicy.
    String[] t = new String[]{ "Kowalski","Nowak","Jażabek"};
    for(String s: t)
        System.out.println(s);
// Wydruk zawartości tablicy.
// Konstrukcja tablicy łańcuchów znakowych za pomocą new.
    double[] b = new double[4];
// Konstrukcja tablicy typu double zawierającej cztery elementy.
    for (int i = 0; i < b.length; i++)
        b[i] = 0.1 * i;
// Nadanie wartości początkowych elementom tablicy.
    for(double x: b)
        System.out.printf(pl,"%g\n",x);
```

W przykładzie podano sposób konstrukcji tablicy liczb zmiennoprzecinkowych i stringów. Wartości elementów tablicy zostają nadane za pomocą pętli for.

```
Wyjście - B5 (run) x
run:
-5
0
10
6
Kowalski
Nowak
Jaźąbek
0,00000
0,100000
0,200000
0,300000
BUILD SUCCESSFUL (total time: 0 seconds)
```

Wczytywanie liczb do tablicy z kontrolą poprawności

Przykład. Poniższy program przedstawia sposób deklaracji dynamicznej dla podanego wymiaru zadania przez użytkownika i sposób wczytania liczb do tablicy z kontrolą poprawności wprowadzanych liczb i wypisuje posortowane liczby.

```
Scanner in = new Scanner(System.in) ;  
int n =0;  
System.out.print("Podaj n:");  
try{  
    n = in.nextInt() ;  
}  
catch(Exception e) {  
    System.out.println("Błąd! "+e) ;  
}
```

//Wczytanie wymiaru zadania z kontrolą poprawności konwersji

```
        if (n < 1){
            System.out.println("Niepoprawna liczba");
            return ;
        }
//Sprawdzenie czy podano poprawny wymiar zadania
        int[] a = new int[n];
//Konstrukcja tablicy
        try{
            for (int i = 0; i < a.length; i++){
//Pętla pobierająca kolejne dane
                System.out.printf("a[%d]=", i);
                a[i]= in.nextInt();
            }
//Sprawdzenie poprawności podanych danych
        }
        catch (Exception e) {
            System.out.println("Błąd! "+e);
        }
    }
```

```
        Arrays.sort(a);  
//Sortowanie elementów w tablicy  
        for (int i = 0; i < a.length; i++)  
            System.out.print(a[i] + " ");  
//Wydruk wprowadzonych liczb  
        System.out.println();
```

Podaj n:4

a[0]=2

a[1]=-1

a[2]=7

a[3]=3

-1 2 3 7

BUILD SUCCESSFUL (total time: 14 seconds)

Podaj n:-1

Niepoprawna liczba

BUILD SUCCESSFUL (total time: 3 seconds)

Podaj n:aaa

Błąd! java.util.InputMismatchException

BUILD SUCCESSFUL (total time: 6 seconds)

Podaj n:3

a[0]=aa

Błąd! java.util.InputMismatchException

BUILD SUCCESSFUL (total time: 5 seconds)

ArrayList

ArrayLista jest obiektem podobnym do tablicy, ale oprócz właściwości przechowującej ilość elementów w obiekcie posiada metody umożliwiające dodawanie elementów do listy oraz metody umożliwiające usuwanie elementy z listy.

UWAGA. Do programu należy zaimportować przestrzeń nazw:

import java.util.;*

Meoda	Opis
boolean add(E element)	Dodaje element na koniec listy
Void add(int index, E element)	Metoda wstawia element pod zadany indeks i przesuwa wszystkie elementy listy.
E get(int index)	Metoda zwraca element z pod pozycji o zadanym indeksie
E remove(int index)	Metoda usuwa element z pod zadanego indeksu listy
E set(int index, E element)	Metoda podstawia zadany element pod indeks listy
int size()	Metoda zwraca liczbę elementów w liście


```
    ArrayList<Integer> a = new ArrayList<>();  
//Konstrukcja listy  
    a.add(4); a.add(9); a.add(7); a.add(5); a.add(-1);  
//Dodanie elementów do listy  
    a.remove(1);  
//Usunięcie elementu znajdującego się po indeksie 1  
    a.add(1, -5);  
//Wstawienie elementu pod zadany indeks i przesunięcie  
//wszystkich pozostałych elementów listy.  
    a.set(1, 6);  
//Wstawienie elementu pod zadany indeks.  
    Collections.sort(a);  
//Sortowanie elementów w liście  
    for (int i = 0; i < a.size(); i++)  
        System.out.print(a.get(i) + " ");  
//Wydruk elementów z listy  
    System.out.println();
```

run:

-1 4 5 6 7

Wczytywanie liczb do listy z kontrolą poprawności

Przykład. Poniższy program przedstawia sposób deklaracji listy sposób wczytania liczb do listy z kontrolą poprawności wprowadzanych liczb i wypisuje największą liczbę.

```
Scanner in = new Scanner(System.in);  
ArrayList<Integer> a = new ArrayList<Integer>();  
//Konstrukcja listy  
String s;  
// Deklaracja łańcucha znaków.  
System.out.print("Podaj a[0]=");
```

```
    while (!(s = in.nextLine()).equals("quit"))
//Wprowadzenie nowej linii i sprawdzenie, czy
//wpisano quit
    try{
        a.add(Integer.parseInt(s));
        System.out.print("Podaj a["+a.size()+"]=");
    }
    catch(Exception e){
        System.out.println("Błąd! "+e);
        return;
    }
    int max = a.get(0);
//Zapisanie pierwszej liczby jako maksimum
    for (int i = 1; i < a.size(); i++)
        if (a.get(i) > max) max = a.get(i);
//Sprawdzenie, czy kolejne liczby nie są większe
    System.out.printf("max=%d\n", max);
//Wypisanie największego elementu
```

Otrzymywane wyniki

```
Wyjście - B8 (run) x
run:
Podań a[0]=3
Podań a[1]=2
Podań a[2]=1
Podań a[3]=9
Podań a[4]=quit
max=9
BUILD SUCCESSFUL (total time: 18 seconds)
```

```
Wyjście - B8 (run) x
run:
Podań a[0]=-1
Podań a[1]=a
Błąd! java.lang.NumberFormatException: For input string: "a"
BUILD SUCCESSFUL (total time: 5 seconds)
```

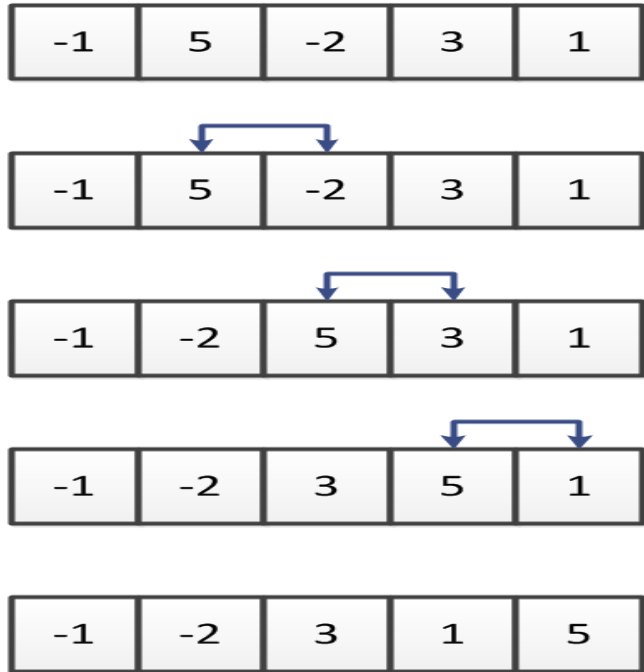
Sortowanie bąbelkowe

Algorytm sortowania bąbelkowego łatwo można rozpisać dla ciągu liczb. Porównujemy w nim ze sobą kolejne elementy szukając ekstremum w każdej rozpatrywanej parze. Znalezioną wartość ekstremalną przesuwamy w ustalonym kierunku. Następnie porównujemy elementy w kolejnej utworzonej parze szukając największego i przesuwając go na koniec. Takie porównania wykonujemy, że dojdziemy do końca ciągu i element największy znajdzie się na ostatnim miejscu. Jak widzimy na poniższych ilustracjach sortowania bąbelkowego, po porównaniu kolejnych liczb w ciągu można dokonać ewentualnej zamiany „sąsiadów”. Ewentualnemu przestawieniu podlega para kolejnych liczb. Jeśli większa z nich jest „pierwszą” w parze zostaje ona wypchnięta na koniec. Następnie w kolejnym kroku analogicznie rozważamy kolejną parę w ciągu.

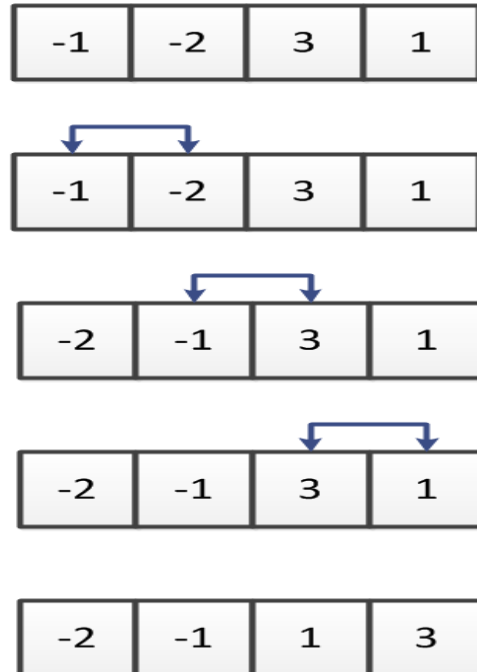
W ten sposób porównamy ze sobą wszystkie pary liczb, jakie znajdują się w rozważanym ciągu. W wyniku każdego kroku algorytmu bąbelkowego element największy zostanie umieszczony dokładnie na końcu rozważanego ciągu. Należy zauważyć, że w każdej iteracji rozważamy ciąg o jeden element mniejszy, ponieważ w wyniku poprzednich kroków wartości maksymalne są ustawione już w kolejności chronologicznej na samym końcu badanego ciągu. Zobaczmy teraz graficzną ilustrację wykonywanych operacji.

Zadanie. Wykonać sortowanie bąbelkowe dla zadanego ciągu liczb -1, 5, -2, 3, 1.

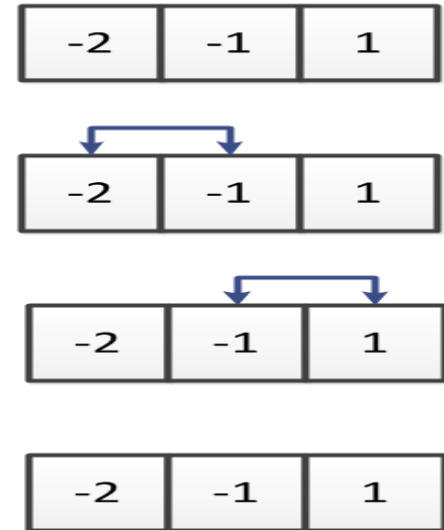
Krok 1:



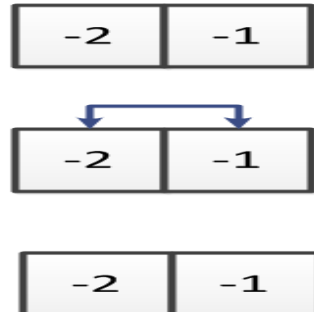
Krok 2:

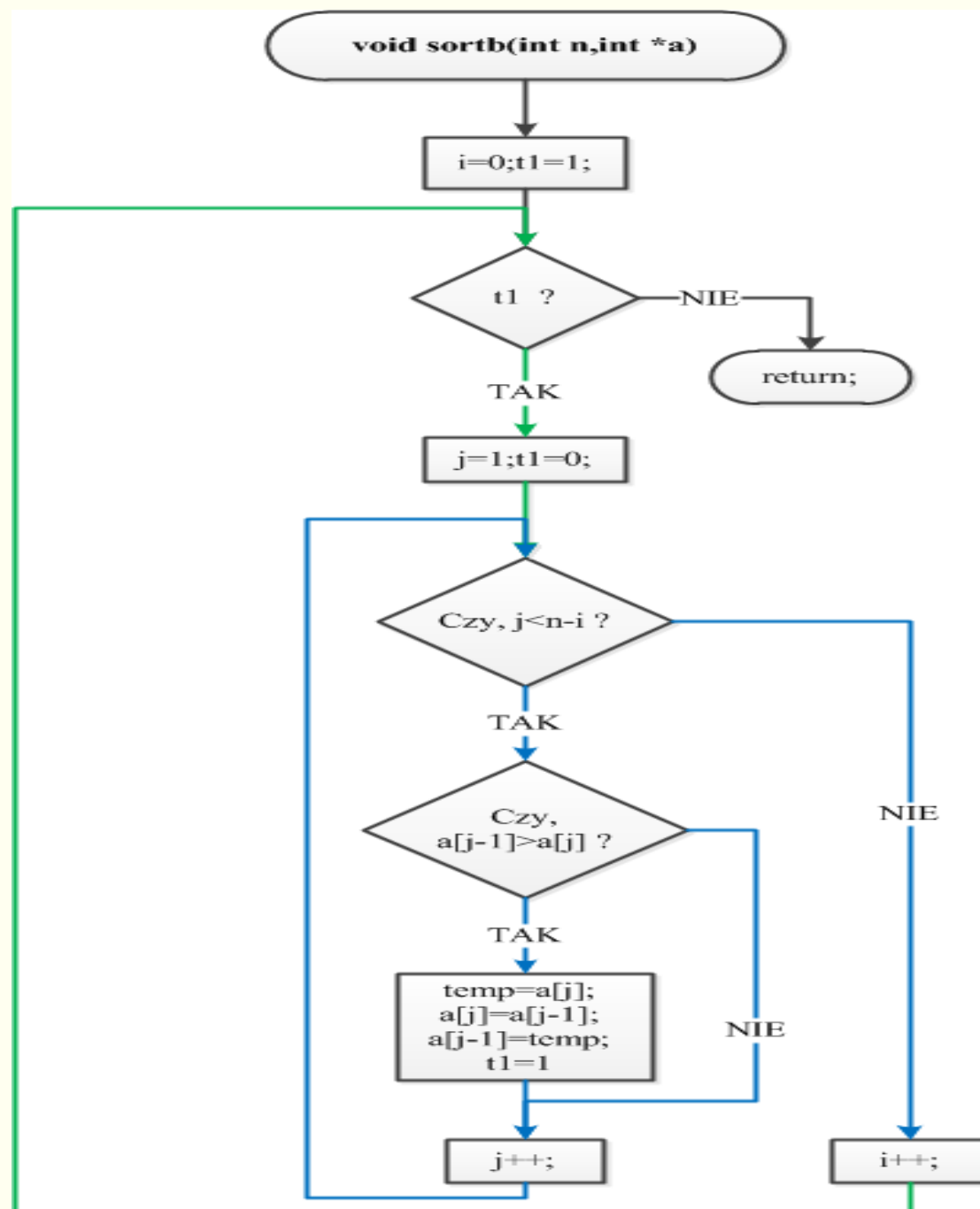


Krok 3:



Krok 4:





Zadanie. Należy posortować liczby znajdujące się w pliku za pomocą sortowania bąbelkowego. W każdej linii została zapisana jedna liczba. Posortowany ciąg liczb należy zapisać do pliku.

```
Scanner in = new Scanner(System.in);  
ArrayList<Integer> a = new ArrayList<Integer>();  
//Konstrukcja listy  
String s = null;  
//Utworzenie pustego łańcucha znaków  
  
System.out.print("Podaj nazwę pliku wejściowego:");  
String s1 = in.nextLine();  
s1 += ".txt";
```


Przeczytanie danych z pliku

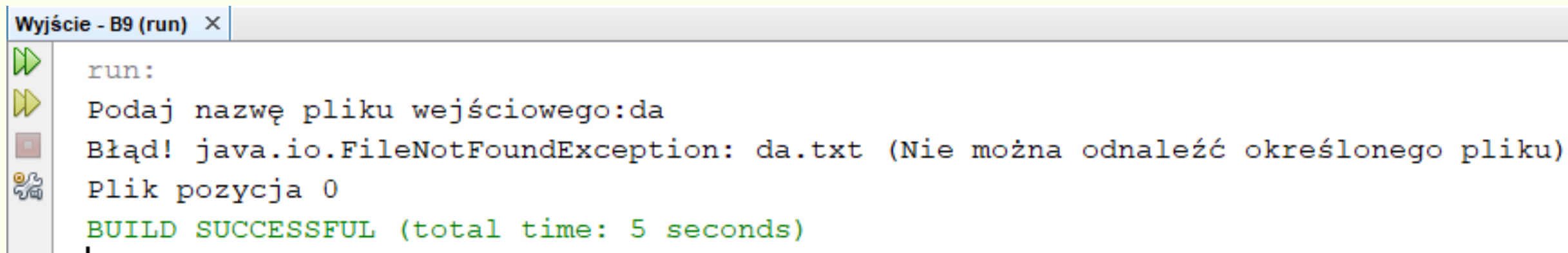
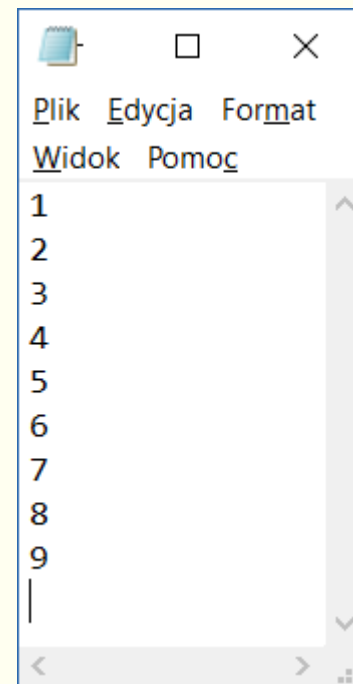
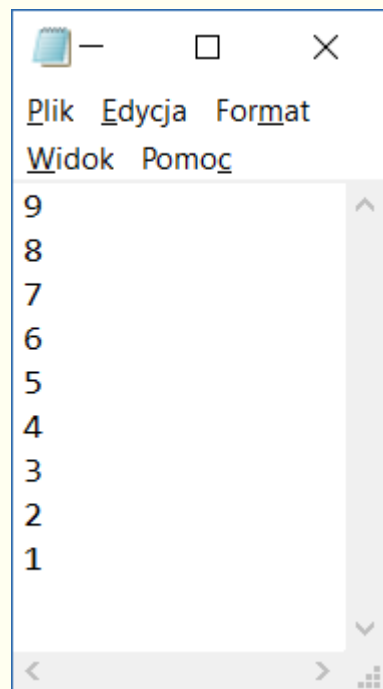
```
try{
    FileReader fr = new FileReader(s1);
    BufferedReader br = new BufferedReader(fr);
    while((s=br.readLine())!=null)
        a.add(Integer.parseInt(s));
    br.close();
}
catch(Exception e){
    System.out.println("Błąd! "+e);
    System.out.println("Plik pozycja "+a.size());
    return;
}
```

Sortowanie bąbelkowe

```
for (int i = 0; i < a.size() - 1; i++)  
    //Sortowanie bąbelkowe  
    for (int j = 1; j < a.size() - i; j++)  
        if (a.get(j - 1) > a.get(j))  
            //Sprawdzenie poprawności ułożenia elementów  
            {  
                int temp = a.get(j);  
                a.set(j, a.get(j-1));  
                a.set(j - 1, temp);  
            }
```

```
System.out.print("Podaj nazwe pliku wyjściowego:");  
String s2 = in.nextLine();  
//Przeczytanie nazwy pliku wyjściowego  
s2 += ".txt";  
try{  
    FileOutputStream fw = new FileOutputStream(s2);  
    PrintWriter wy = new PrintWriter(fw);  
    for (int i = 0; i < a.size(); i++)  
        //Zapisanie posortowanego zbioru liczb do pliku  
        wy.println(a.get(i));  
    wy.close();  
}  
catch (Exception e) {  
    System.out.println("Błąd! "+e);  
    return;  
}
```

Uruchomienie programu



Zadanie. Napisać program znajdujący rozwiązanie układu równań liniowych postaci:

$$\begin{cases} ax + by = c \\ dx + ey = f \end{cases}$$

, gdzie współczynniki a, b, c, d, e, f są zadanymi liczbami rzeczywistymi.

Do rozwiązywania tego przykładu zastosujemy metodę wyznaczników znaną Czytelnikowi z kursu algebry. Metodę tę dla przypomnienia zdefiniujemy raz jeszcze. Na początku wyznaczamy następujące wyznaczniki układu równań liniowych.

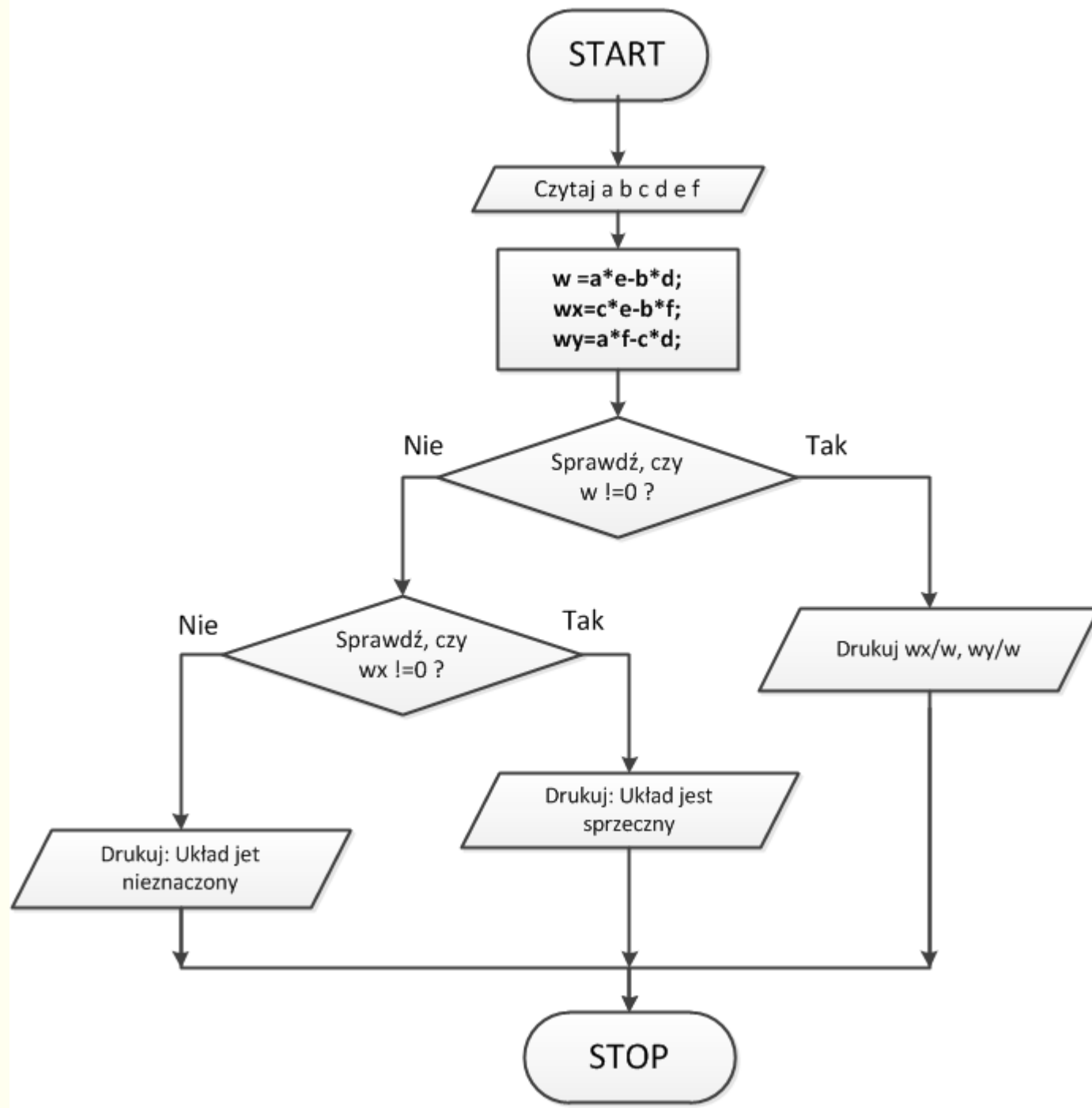
$$W = \begin{vmatrix} a & b \\ d & e \end{vmatrix} = a * e - b * d$$

$$W_x = \begin{vmatrix} c & b \\ f & e \end{vmatrix} = c * e - b * f$$

$$W_y = \begin{vmatrix} a & c \\ d & f \end{vmatrix} = a * f - c * d$$

Mówimy wtedy o następujących możliwościach:

- 1) Jeżeli $W \neq 0$ to układ reprezentowany wyznacznikami jest oznaczony oraz istnieje jedna i tylko jedna para liczb $x = \frac{W_x}{W}$ oraz $y = \frac{W_y}{W}$ spełniająca ten układ równań.
- 2) Jeżeli $W = 0$ oraz $W_x \neq 0$, co pociąga za sobą, że $W_y \neq 0$, to układ reprezentowany wyznacznikami jest sprzeczny oraz żadna para liczb nie spełnia tego układu równań.
- 3) Jeżeli $W = 0$ oraz $W_x = 0$, co pociąga za sobą, że $W_y = 0$, to układ reprezentowany wyznacznikami jest nieoznaczony oraz istnieje nieskończenie wiele par liczb spełniających ten układ równań.



Zapiszmy teraz
przedstawioną
metodę w
postaci
schematu
blokowego, a
następnie na
jego podstawie
w postaci kodu
programu.

```
Scanner in = new Scanner(System.in);
System.out.printf("Podaj a b c d e f\n");
double a = in.nextDouble();
double b = in.nextDouble();
double c = in.nextDouble();
double d = in.nextDouble();
double e = in.nextDouble();
double f = in.nextDouble();
double W = a * e - b * d;
double Wx = c * e - b * f;
double Wy = a * f - c * d;
/* Wyznaczenie wyznaczników układu równań
   zgodnie z przedstawionymi wzorami */
```



```
        if (W != 0)
/* Instrukcja warunkowa if sprawdzająca
   wartość wyznacznika głównego*/
    {
        System.out.printf("x1=%g\n", Wx / W);
        System.out.printf("x2=%g\n", Wy / W);
    }
else
if (Wx != 0)
    /* Instrukcja warunkowa if sprawdzająca
       wartość wyznacznika Wx*/
    System.out.println
        ("Układ równan jest sprzeczny");
else
    System.out.println
        ("Układ równan jest nieoznaczony");
```

Sprawdzenie działania programu

```
run:
Podaj a b c d e f
1,0
2,0
3,0
2
1
3|
x1=1,00000
x2=1,00000
```

```
run:
Podaj a b c d e f
0
0
0
0
0
0
0
Układ równan jest nieoznaczony
```

```
run:
Podaj a b c d e f
1
2
3
1
2
4
Układ równan jest sprzeczny
```

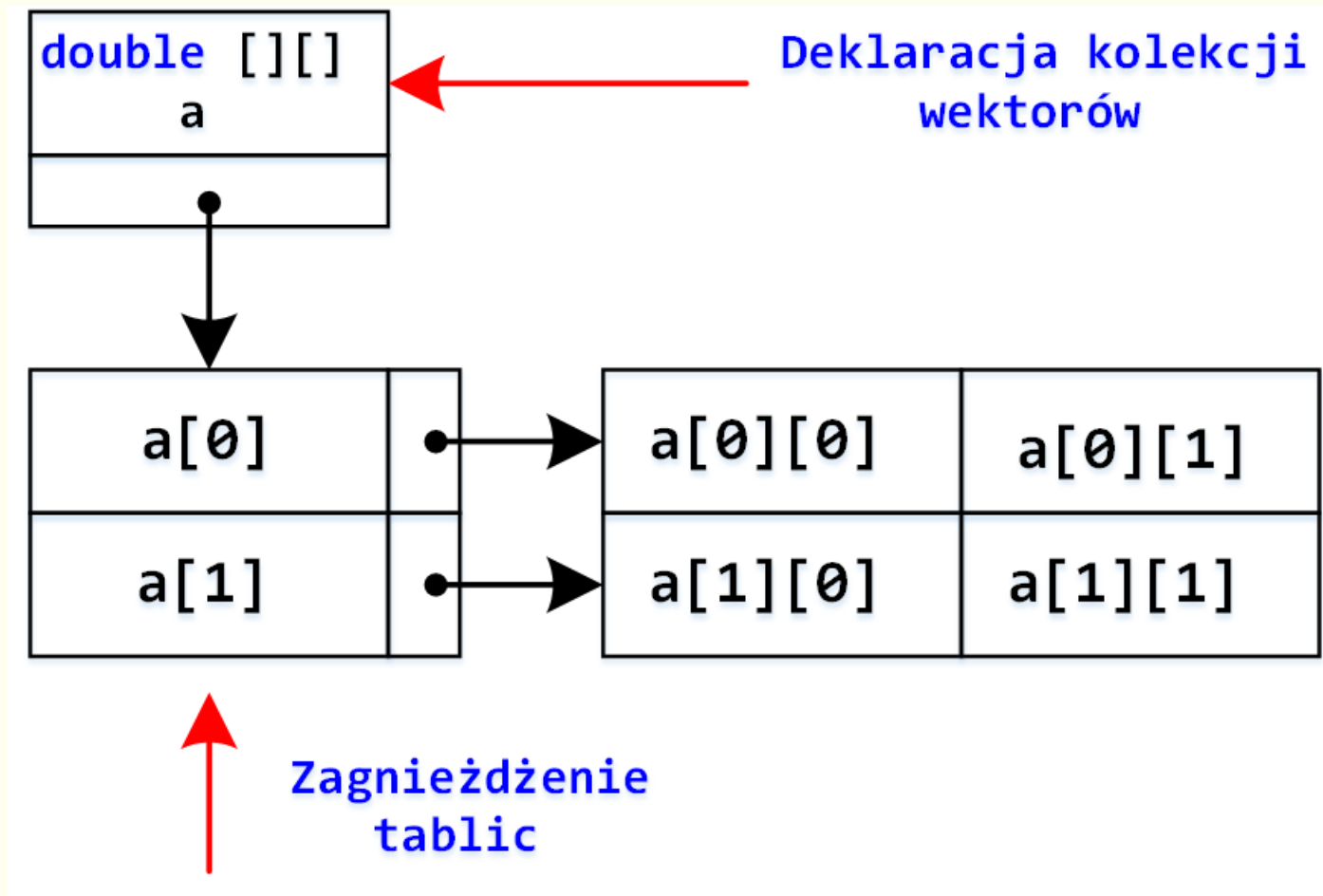
Tablice postrzępione

Elementy tablic mogą być uchwytami do kolejnych tablic. Tak zagnieżdżone tablice nazywamy tablicami postrzępionymi (ang. *jagged arrays*). Poparzymy na ich konstrukcję:

```
double [][] a= new double[2][];  
    a[0]= new double[2];  
    a[1]= new double[2];
```

Definiuje ona tablicę dwuwymiarową o dwóch wierszach i dwóch kolumnach. Sposób konstrukcji tablicy ilustruje Rys.

Konstrukcja tablicy dwuwymiarowej



Rozmieszczenie danych w tablicy przy użyciu kreatora new.

Wykorzystajmy przedstawiony sposób konstrukcji tablicy dwuwymiarowej do napisania programu znajdującego rozwiązanie układu równań liniowych:

$$\begin{cases} a_{00}x_0 + a_{01}x_1 = b_0 \\ a_{10}x_0 + a_{11}x_1 = b_1 \end{cases}$$

Układ równań możemy zapisać w postaci macierzowej:

$$\begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$$

lub prościej, jako:

$$Ax = b, \text{ gdzie } A \in R^{2 \times 2} \text{ i } x, b \in R^2$$

Kod naszego programu wygląda następująco:

```
Scanner in = new Scanner(System.in);
double [][] a = new double[2][2];
// Deklaracja macierzy A.
double [] b = new double[2];
// Deklaracja wektora b.
for (int i = 0; i < 2; i++)
{
    for (int j = 0; j < 2; j++)
    {
        System.out.printf("Podaj a[%d][%d]:", i, j);
        a[i][j] = in.nextDouble();
        // Wczytanie elementów macierzy A.
    }
    System.out.printf("Podaj b[%d]:", i);
// Wczytanie elementów wektora b.
    b[i] = in.nextDouble();
}
```

```
double W = a[0][0] * a[1][1] - a[0][1] * a[1][0];
double Wx = b[0] * a[1][1] - a[0][1] * b[1];
double Wy = a[0][0] * b[1] - b[0] * a[1][0];
//Wyznaczenie wyznaczników układu
if (W != 0)
// Sprawdzenie kolejnych warunków rozwiązalności
// układu.
{
    System.out.printf("x[0] = %g\n", Wx / W);
    System.out.printf("x[1] = %g\n", Wy / W);
}
else if (Wx != 0)
    System.out.println
        ("Układ równań jest sprzeczny");
else
    System.out.println
        ("Układ równań jest nieoznaczony");
```

Sprawdzenie działania programu

```
Podaj a[0][0]:1,0|
Podaj a[0][1]:2,0
Podaj b[0]:3,0
Podaj a[1][0]:2
Podaj a[1][1]:1
Podaj b[1]:3
x[0] = 1,00000
x[1] = 1,00000
```

```
Podaj a[0][0]:0
Podaj a[0][1]:0
Podaj b[0]:0
Podaj a[1][0]:0
Podaj a[1][1]:0
Podaj b[1]:0
Układ równań jest nieoznaczony
```

```
Podaj a[0][0]:1
Podaj a[0][1]:2
Podaj b[0]:3
Podaj a[1][0]:1
Podaj a[1][1]:2
Podaj b[1]:4
Układ równań jest sprzeczny
```


Alokacja różnych wymiarów tablicy dla drugiego wymiaru

```
int a[][] = new int[4][];  
for(int i=0; i < 4; i++)  
a[i] = new int[i+1];  
  
int k = 0;  
for(int i=0; i<4;i++)  
    for(int j=0; j<i+1;j++)  
        a[i][j] = k++;  
for(int i=0; i<4;i++){  
    for(int j=0; j<i+1;j++)  
        System.out.print(a[i][j]+" ");  
    System.out.println();  
}
```

Wyjście - B2f (run) X

run:

0

1 2

3 4 5

6 7 8 9

Czytanie liczb rzeczywistych z pliku za pomocą skanera

```
int c1 = 0, c2 = 0;
double sum = 0.0;
// System.setProperty("file.encoding", "UTF-8");
// Ustawienie strony kodowej programu
try{
    FileReader fin = new FileReader("dane.txt");
    Scanner src = new Scanner(fin);
// Utworzenie skanera czytającego plik
    while(src.hasNext()){
        if(src.hasNextDouble())
// Sprawdzenie, czy następny element jest liczbą
        { double d = src.nextDouble();
          System.out.printf("%g\n", d);
          sum+=d;
        }
    }
}
```

```
        else{
            String s = src.next();
            System.out.println("String: "+s);
            c2++;
        }
    }
}

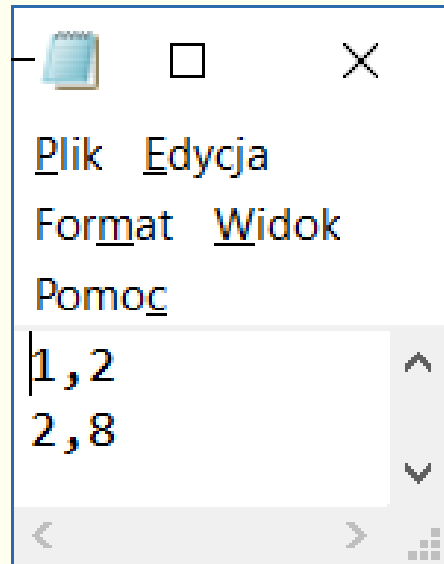
catch (Exception e) {
    System.out.println("Błąd! "+e);
}

System.out.printf("Liczba el. %d suma = %g\n",c1,sum );
System.out.printf("Liczba łańcuchów = %d\n", c2);
```

Wykonanie programu

Plik zapisany w kodowaniu

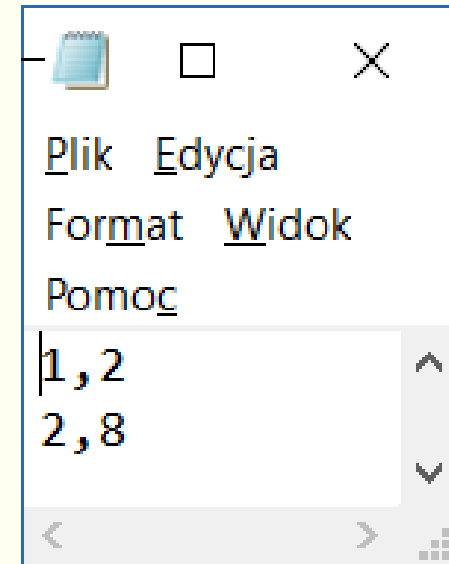
ASCII



```
run:
1,20000
2,80000
Liczba el. 0 suma = 4,00000
Liczba łańcuchów = 0
```

Plik zapisany w kodowaniu

UTF-8



```
run:
String: 1,2
2,80000
Liczba el. 0 suma = 2,80000
Liczba łańcuchów = 1
```